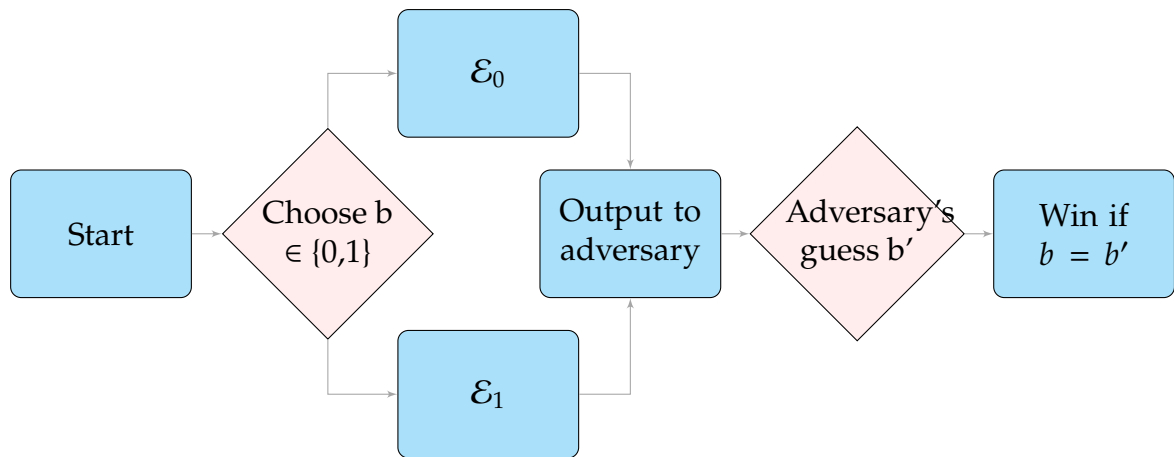


Provable Security in Cryptography

Ji Yong-Hyeon, Kim Dong-Hyeon



Department of Information Security, Cryptology, and Mathematics
College of Science and Technology
Kookmin University

November 16, 2023

Contents

1	One-Time Pad & Kerckhoff's Principle	2
2	The Basics of Provable Security	4
2.1	How to Write a Security Definition	4
2.1.1	Syntax and Correctness	4
2.1.2	"Real-vs-Random" Style of Security Definition	5
2.1.3	"Left-vs-Right" Style of Security Definition	6
2.2	Formalisms for Security Definition	6
2.3	How to Demonstrate Insecurity with Attacks	7
2.4	How to Prove Security with The Hybrid Technique	7
2.5	How to Compare/Contract Security Definitions	7
3	Cryptography on Intractable Computations	9
3.1	What Qualifies as a "Computationally Infeasible" Attack?	9
3.2	What Qualifies as a "Negligible" Success Probability?	10
3.3	Indistinguishability	12
3.4	Birthday Probabilities & Sampling With/out Replacement	15
4	Pseudo-random Generators (PRG)	20
4.1	Definition	20
4.2	Shorter Keys in One-Time-Secret Encryption	25
5	Pseudo-Random Functions & Block Ciphers	27
5.1	Definition	28

List of Symbols

$0^\lambda, 1^\lambda$ $\underbrace{00 \dots 0}_{\lambda \text{ times}}, \underbrace{11 \dots 1}_{\lambda \text{ times}} : \lambda\text{-bit zero/one sequence}$

$\mathcal{A} \diamond \mathcal{L}$ The result of **linking** $\mathcal{A}$ to $\mathcal{L}$

$\$$ Randomeness

$\equiv$ Interchangability; Identical

Chapter 1

One-Time Pad & Kerckhoff's Principle

Kerckhoffs' Principle:

Design your system to be secure even if the attacker has complete knowledge of all its algorithms.

One-time Pad (OTP)

Construction 1.1. The specific KeyGen, Enc, and Dec algorithms for **one-time pad** are given below:

KeyGen :	Enc($k, m \in \{0, 1\}^\lambda$) :	Dec($k, c \in \{0, 1\}^\lambda$) :
$k \xleftarrow{\$} \{0, 1\}^\lambda$ return k	return $k \oplus m$	return $k \oplus c$

Correctness of OTP

Proposition 1.1.

$$(\forall k, m \in \{0, 1\}^\lambda) \quad \text{Dec}(k, \text{Enc}(k, m)) = m.$$

Proof. Let $k, m \in \{0, 1\}^\lambda$ then

$$\begin{aligned} \text{Dec}(k, \text{Enc}(k, m)) &= \text{Dec}(k, k \oplus m) = k \oplus (k \oplus m) \\ &= (k \oplus k) \oplus m \\ &= 0^\lambda \oplus m \\ &= m. \end{aligned}$$

□

Remark 1.1 (Eavesdrop Algorithm). From Eve's perspective, seeing a ciphertext corresponds to receiving an output from the following algorithm:

Eavesdrop($m \in \{\mathbf{0}, \mathbf{1}\}^\lambda$)

$k \xleftarrow{\$} \{\mathbf{0}, \mathbf{1}\}^\lambda$
 $c := k \oplus m$
 return c

Theorem 1.2. Let $m \in \{\mathbf{0}, \mathbf{1}\}^\lambda$. The distribution $\text{Eavesdrop}(m)$ is the *uniform distribution* on $\{\mathbf{0}, \mathbf{1}\}^\lambda$. In other words,

$$m, m' \in \{\mathbf{0}, \mathbf{1}\}^\lambda \implies \text{dist}(\text{Eavesdrop}(m)) \sim \text{dist}(\text{Eavesdrop}(m')).$$

Proof. content...

□

Chapter 2

The Basics of Provable Security

2.1 How to Write a Security Definition

2.1.1 Syntax and Correctness

Encryption Syntax

Definition 2.1. A symmetric-key encryption (SKE) scheme consists of the following algorithms:

- KeyGen outputs a key $k = \text{KeyGen}(1^\lambda) \in \mathcal{K}$
- $\text{Enc} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$
- $\text{Dec} : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$

We call $\mathcal{K}$ the **key space**, $\mathcal{M}$ the **message space**, and $\mathcal{C}$ the **ciphertext space** of the scheme.

Remark 2.1. Note that

- KeyGen is a randomized algorithm¹.
- Enc is a (possibly randomized) algorithm².
- Dec is a deterministic algorithm³.

Remark 2.2. We refer to the entire scheme by a single variable Σ , i.e.,

$$\Sigma = (\text{KeyGen}, \text{Enc}, \text{Dec}).$$

Remark 2.3. We write

$$\begin{array}{ccc} \Sigma.\text{KeyGen}, & \Sigma.\text{Enc}, & \Sigma.\text{Dec}, \\ \Sigma.\mathcal{K}, & \Sigma.\mathcal{M}, & \Sigma.\mathcal{C} \end{array}$$

to refer to its components.

¹An algorithm that makes use of random numbers.

²It could operate deterministically or non-deterministically depending on specific conditions or parameters.

³An algorithm that does produces the same output for the same input, every time it's run.

SKE Correctness

Definition 2.2. An encryption scheme Σ satisfies **correctness** if

$$(\forall k \in \Sigma.\mathcal{K}) (\forall m \in \Sigma.\mathcal{M}) \quad \Pr [\Sigma.\text{Dec}(k, \Sigma.\text{Enc}(k, m)) = m] = 1.$$

Remark 2.4. The definition is written in terms of a probability because Enc is allowed to be a randomized algorithm. In other words, decrypting a ciphertext with the same key that was used for encryption must *always* result in the original plaintext.

Example 2.1. content...

2.1.2 “Real-vs-Random” Style of Security Definition

“an encryption scheme is a good one if its ciphertexts *look like* random junk to an attacker”

Security definitions always consider **the attacker’s view** of the system.

“an encryption scheme is a good one if its ciphertexts *look like* random junk to an attacker ... when each key is secret and used to encrypt only one plaintext, even when the attacker chooses the plaintexts.”

A concise way to express all of these details is to consider **the attacker as a calling program** to the following subroutine:

$\begin{array}{l} \text{CTXT}(m \in \Sigma.\mathcal{M}) : \\ \hline k \leftarrow \Sigma.\text{KeyGen} \\ c := \Sigma.\text{Enc}(k, m) \\ \text{return } c \end{array}$
--

Example 2.2 (One-Time Pad (OTP)). a

$\begin{array}{l} \text{CTXT}(m) : \\ \hline k \leftarrow \{\mathbf{0}, \mathbf{1}\}^\lambda \quad // \text{KeyGen of OTP} \\ c := k \oplus m \quad // \text{Enc of OTP} \\ \text{return } c \end{array}$

vs.

$\begin{array}{l} \text{CTXT}(m) : \\ \hline c := \{\mathbf{0}, \mathbf{1}\}^\lambda \quad // C \text{ of OTP} \\ \text{return } c \end{array}$

“an encryption scheme is a good one if, when you plug its KeyGen and Enc algorithms into the template of the CTXT subroutine above, the two implementations of CTXT induce identical behavior in every calling program.”

2.1.3 “Left-vs-Right” Style of Security Definition

2.2 Formalisms for Security Definition

Library

Definition 2.3. A library $\mathcal{L}$ is a collection of subroutines and private/static variables.

Example 2.3. Here is a familiar library and one possible calling program:

$\mathcal{L}$		$\mathcal{A}$
$\text{CTXT}(m):$ $k \xleftarrow{\$} \{\mathbf{0}, \mathbf{1}\}^\lambda$ $c := k \oplus m$ return c	,	$m \leftarrow \{\mathbf{0}, \mathbf{1}\}^\lambda$ $c := \text{CTXT}(m)$ return $m \stackrel{?}{=} c$

Then

$$\Pr[\mathcal{A} \diamond \mathcal{L} \Rightarrow \text{true}] = \frac{1}{2^\lambda}.$$

Interchangeability

Definition 2.4. Let $\mathcal{L}_1$ and $\mathcal{L}_2$ be two libraries that have the same interface. We say that $\mathcal{L}_1$ and $\mathcal{L}_2$ are **interchangeable**, and write $\mathcal{L}_1 \equiv \mathcal{L}_2$, if $\forall \mathcal{A}$:

$$\Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow \text{true}] = \Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow \text{true}].$$

One-Time Uniform Ctxts

Definition 2.5. An encryption scheme Σ has **one-time uniform cipher-texts** if

$\mathcal{L}_{\text{ots\$-real}}^\Sigma$		$\mathcal{L}_{\text{ots\$-rand}}^\Sigma$
$\text{CTXT}(m \in \Sigma.\mathcal{M}):$ $k \leftarrow \Sigma.\text{KeyGen}$ $c \leftarrow \Sigma.\text{Enc}(k, m)$ return c	$\equiv$	$\text{CTXT}(m \in \Sigma.\mathcal{M}):$ $c \leftarrow \Sigma.\mathcal{C}$ return c

One-Time Secrecy (OTS)

Definition 2.6. **One-time secrecy** is a property of an encryption scheme where an adversary cannot gain any information about the plaintext message from the ciphertext, even if they know the encryption key was used only once.

$$\begin{array}{|c|} \hline \mathcal{L}_{\text{ots-L}}^\Sigma \\ \hline \text{Eve}(m_L, m_R \in \Sigma.\mathcal{M}): \\ \hline k \leftarrow \Sigma.\text{KeyGen} \\ c \leftarrow \Sigma.\text{Enc}(k, m_L) \\ \text{return } c \\ \hline \end{array} \quad \equiv \quad \begin{array}{|c|} \hline \mathcal{L}_{\text{ots-R}}^\Sigma \\ \hline \text{Eve}(m_L, m_R \in \Sigma.\mathcal{M}): \\ \hline k \leftarrow \Sigma.\text{KeyGen} \\ c \leftarrow \Sigma.\text{Enc}(k, m_R) \\ \text{return } c \\ \hline \end{array}$$

2.3 How to Demonstrate Insecurity with Attacks

2.4 How to Prove Security with The Hybrid Technique

2.5 How to Compare/Contract Security Definitions

Exercises

Exercise 2.1. In abstract algebra, a (finite) group is a finite set $\mathbb{G}$ of items together with an operator $\otimes$ satisfying the following axioms:

- **Closure:** for all $a, b \in \mathbb{G}$, we have $a \otimes b \in \mathbb{G}$
- **Identity:** there is a special *identity element* $e \in \mathbb{G}$ that satisfies $e \otimes a = a \otimes e = a$ for all $a \in \mathbb{G}$. We typically write “1” rather than e for the identity element.
- **Associativity:** for all $a, b, c \in \mathbb{G}$, we have $(a \otimes b) \otimes c = a \otimes (b \otimes c)$.
- **Inverses:** for all $a \in \mathbb{G}$, there exists an inverse element $b \in \mathbb{G}$ such that $a \otimes b = b \otimes a$ is the identity element of $\mathbb{G}$. We typically write “ a^{-1} ” for the inverse of a .

Define the following encryption scheme in terms of an arbitrary group $(\mathbb{G}, \otimes)$:

$\mathcal{K} = \mathbb{G}$	<u>KeyGen :</u>	<u>Enc(k, m) :</u>	<u>Dec(k, c) :</u>
$\mathcal{M} = \mathbb{G}$	$k \leftarrow \mathbb{G}$	return $k \otimes m$	??
$\mathcal{C} = \mathbb{G}$	return k		

- Prove that $\{\mathbf{0}, \mathbf{1}\}^\lambda$ is a group with respect to the xor operator. What is the identity element, and what is the inverse of a value $x \in \{\mathbf{0}, \mathbf{1}\}^\lambda$?
- Fill in the details of the Dec algorithm and prove (using the group axioms) that the scheme satisfies correctness.
- Prove that the scheme satisfies one-time secrecy.

Exercise 2.2. Prove that if an encryption scheme Σ has $|\Sigma.\mathcal{K}| < |\Sigma.\mathcal{M}|$ then it cannot satisfy one-time secrecy.

[Hint: The definition of interchangeability does not place any restriction on the running time of the distinguisher/calling program. Even an exhaustive brute-force attack would be valid]

Solution. content...

□

Chapter 3

Cryptography on Intractable Computations

3.1 What Qualifies as a “Computationally Infeasible” Attack?

Polynomial Time

Definition 3.1. A program runs in **polynomial time** if

$$\exists c > 0 : \forall n \geq n_0 : \text{Time}(n) \leq n^c,$$

where Time is the time taken by the algorithm on inputs of size n . n_0 is constant size of the input. That is, there exists a constant $c > 0$ such that for all sufficiently long input strings x with $|x| = n$, the program stops after no more than $O(n^c)$ steps.

Remark 3.1. We see “polynomial-time” as a synonym for “efficient.”

Example 3.1. $\text{gcd}(a, b)$ can be computed using $O((\log_2 a)^3)$ bit operation if $a > b$.

Example 3.2.

Efficient algorithm known:	No known efficient algorithm:
Computing GCDs	Factoring integers
Arithmetic mod N	Computing $\phi(N)$ given N
Inverses mod N	Discrete logarithm
Exponentiation mod N	Square roots mod composite N

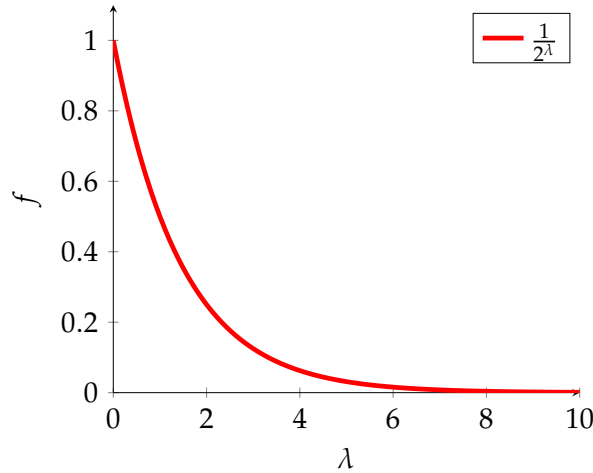
Again, “efficient” means polynomial-time. Furthermore, we only consider polynomial-time algorithms that run on standard, *classical* computers. In fact, all of the problems in the right-hand column *do* have known polynomial-time algorithms on *quantum* computers.

3.2 What Qualifies as a “Negligible” Success Probability?

For a cryptographic system to be considered secure, we often want the success probability of any polynomial-time adversary to be negligible in the security parameter λ .

Idea. $\frac{1}{2^\lambda}$ approaches zero so fast that no polynomial can “rescue”.

Proof. Assume that $f(\lambda) = \frac{1}{2^\lambda}$.



Consider any polynomial $p(\lambda)$ of degree n , written as:

$$p(\lambda) = a_0 + a_1\lambda + a_2\lambda^2 + \cdots + a_n\lambda^n = \sum_{i=0}^n a_i\lambda^i.$$

The product $p(\lambda)$ and $f(\lambda)$ is

$$p(\lambda)f(\lambda) = a_0\frac{1}{2^\lambda} + a_1\frac{\lambda}{2^\lambda} + \cdots + a_n\frac{\lambda^n}{2^\lambda}.$$

We claim that $\lim_{\lambda \rightarrow \infty} a_k \frac{\lambda^k}{2^\lambda} = 0$, where $k \in \mathbb{Z}_{\geq 0}$. Let $g(\lambda) = \lambda^k$ and $h(\lambda) = 2^\lambda$. Note that

$$\begin{aligned} h'(\lambda) &= 2^\lambda(\ln 2), & g'(\lambda) &= k\lambda^{k-1} \\ h''(\lambda) &= 2^\lambda(\ln 2)^2, & g''(\lambda) &= k(k-1)\lambda^{k-2} \\ &\vdots & & \\ h^{(k)}(\lambda) &= 2^\lambda(\ln 2)^k, & g^{(k)}(\lambda) &= k!. \end{aligned}$$

By applying L'Hôpital's Rule k times, we have

$$\lim_{\lambda \rightarrow \infty} \frac{\lambda^k}{2^\lambda} = \lim_{\lambda \rightarrow \infty} \frac{k!}{2^\lambda(\ln 2)^k} = 0.$$

Thus, $\lim_{\lambda \rightarrow \infty} p(\lambda)f(\lambda) = 0$. □

Negligible

Definition 3.2. A function f is **negligible** if,

$$\forall \text{polynomial } p : \lim_{\lambda \rightarrow \infty} p(\lambda)f(\lambda) = 0.$$

In other words, a negligible function approaches zero so fast that you can never catch up when multiplying by a polynomial.

Remark 3.2. As λ (security parameter) gets larger and larger, the product of $p(\lambda)$ (resources or capabilities for an adversary) and $f(\lambda)$ (success probability) approaches 0.

Remark 3.3. A function $f(\lambda)$ is negligible if $\forall p(\lambda) > 0 : \exists \lambda_0 : \lambda > \lambda_0 \Rightarrow |f(\lambda)| < \frac{1}{p(\lambda)}$.

Proposition 3.1. Let $c \in \mathbb{Z}$.

$$\lim_{\lambda \rightarrow \infty} \lambda^c f(\lambda) = 0 \implies f \text{ is negligible.}$$

Proof. Suppose that f satisfies $\lim_{\lambda \rightarrow \infty} \lambda^c f(\lambda) = 0$ for any $c \in \mathbb{Z}$, and take an arbitrary polynomial p of degree n . Since $\lim_{\lambda \rightarrow \infty} \frac{p(\lambda)}{\lambda^{n+1}} = 0$, we have

$$\lim_{\lambda \rightarrow \infty} p(\lambda)f(\lambda) = \lim_{\lambda \rightarrow \infty} \left[\frac{p(\lambda)}{\lambda^{n+1}} \left(\lambda^{n+1} \cdot f(\lambda) \right) \right] = \left(\lim_{\lambda \rightarrow \infty} \frac{p(\lambda)}{\lambda^{n+1}} \right) \left(\lim_{\lambda \rightarrow \infty} \lambda^{n+1} f(\lambda) \right) = 0 \cdot 0 = 0.$$

□

Example 3.3. Let $c \in \mathbb{Z}$. Then

$$\lim_{\lambda \rightarrow \infty} \lambda^c \frac{1}{2^\lambda} = \lim_{\lambda \rightarrow \infty} \frac{(\lambda^c)^{\log_2 2}}{2^\lambda} = \lim_{\lambda \rightarrow \infty} \frac{2^{c \log_2 \lambda}}{2^\lambda} = \lim_{\lambda \rightarrow \infty} 2^{c \log_2(\lambda) - \lambda} = 0$$

since $c \log_2(\lambda) - \lambda \rightarrow -\infty$ as $\lambda \rightarrow \infty$. Thus, $1/2^\lambda$ is negligible.

 $f \approx g$

Definition 3.3. Let $f, g : \mathbb{N} \rightarrow \mathbb{R}$ are real-valued functions. We write $f \approx g$ to mean that $|f(\lambda) - g(\lambda)|$ is a negligible function.

Remark 3.4. We use the terminology of negligible functions exclusively when discussing probabilities, so the following are common:

$\Pr[X] \approx 0 \Leftrightarrow$ “event X almost never happens”

$\Pr[Y] \approx 1 \Leftrightarrow$ “event Y almost always happens”

$\Pr[A] \approx \Pr[B] \Leftrightarrow$ “event A and B happen with essentially the same probability”

Additionally, the $\approx$ symbol is *transitive*:

$$\Pr[X] \approx \Pr[Y] \wedge \Pr[Y] \approx \Pr[Z] \implies \Pr[X] \approx \Pr[Z].$$

3.3 Indistinguishability

Indistinguishable ($\approx$)

Definition 3.4. Let $\mathcal{L}_1$ and $\mathcal{L}_2$ be two libraries with a common interface, and let $\mathcal{A}$ is a polynomial-time program that output a single bit. We say that $\mathcal{L}_1$ and $\mathcal{L}_2$ are **indistinguishable**, and write $\mathcal{L}_1 \approx \mathcal{L}_2$, if

$$\Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow 1] \approx \Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1].$$

Remark 3.5.

(1) We call the quantity

$$|\Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow 1] - \Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1]|$$

the **advantage** (or **bias**) of $\mathcal{A}$ in distinguishing $\mathcal{L}_1$ and $\mathcal{L}_2$.

(2) Two libraries are indistinguishable if all polynomial-time calling programs have negligible advantage in distinguishing them.

Example 3.4. Two indistinguishable libraries:

$\mathcal{L}_1$	$\mathcal{L}_2$
$\begin{array}{l} \text{Predict}(x): \\ \hline s \xleftarrow{\$} \{\mathbf{0}, \mathbf{1}\}^\lambda \\ \text{return } x \stackrel{?}{=} s \end{array}$	$\begin{array}{l} \text{Predict}(x): \\ \hline \text{return false} \end{array}$

The calling program $\mathcal{A}$ repeatedly invokes the ‘Predict’ functions and returns ‘1’ if it ever obtains a ‘true’ value from the response:

$\mathcal{A}$
$\begin{array}{l} \text{do } q \text{ times:} \\ \quad \text{if Predict}(\mathbf{0}^\lambda) = \text{true} \\ \quad \quad \text{return 1} \\ \text{return 0} \end{array}$

Then

(1) $\mathcal{L}_2$ can never return true, i.e.,

$$\Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1] = 0.$$

(2) $\Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1]$ is surely non-zero.

$$\begin{aligned} \Pr[\mathcal{A} \diamond \Rightarrow 1] &= 1 - \Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow 0] \\ &= 1 - \left(1 - \frac{1}{2^\lambda}\right)^q. \end{aligned}$$

Using the union bound, we get:

$$\begin{aligned} \Pr[\mathcal{A} \diamond \Rightarrow 1] &\leq \Pr[\text{first call to Predict returns true}] \\ &\quad + \Pr[\text{second call to Predict returns true}] \\ &\quad + \dots \\ &= q \cdot \frac{1}{2^\lambda}. \end{aligned}$$

We showed that $\mathcal{A}$ has non-zero advantage, and so $\mathcal{L}_1 \neq \mathcal{L}_2$. We also showed that $\mathcal{A}$ has advantage at most $q/2^\lambda$. Since $\mathcal{A}$ runs in polynomial time, it can only make a polynomial number q of queries to the library, so $q/2^\lambda$ is negligible.

Lemma 3.2.

- (1) $\mathcal{L}_1 \equiv \mathcal{L}_2 \implies \mathcal{L}_1 \approx \mathcal{L}_2$.
- (2) $\mathcal{L}_1 \approx \mathcal{L}_2 \approx \mathcal{L}_3 \implies \mathcal{L}_1 \approx \mathcal{L}_3$.

Proof. content...

□

Lemma 3.3. For any polynomial-time library $\mathcal{L}^*$,

$$\mathcal{L}_1 \approx \mathcal{L}_2 \implies \mathcal{L}^* \diamond \mathcal{L}_1 \approx \mathcal{L}^* \diamond \mathcal{L}_2.$$

Proof. content...

□

Bad-Event Lemma

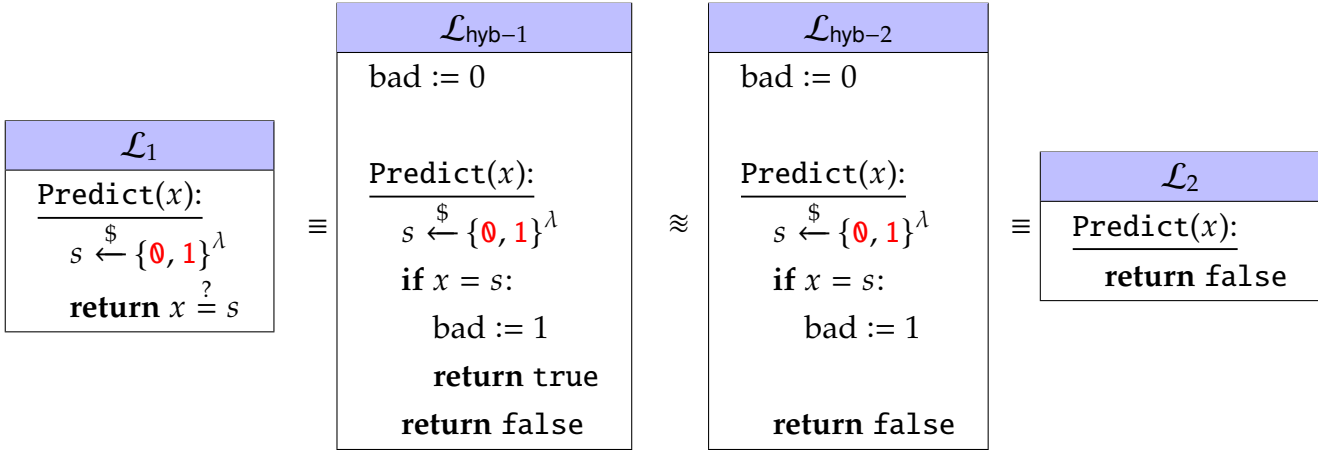
Lemma 3.4.

$$|\Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow 1] - \Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1]| \leq \Pr[\mathcal{A} \diamond \mathcal{L}_1 \text{ sets bad} = 1].$$

Proof.

□

Example 3.5. Consider $\mathcal{L}_1$ and $\mathcal{L}_2$. They are indistinguishable with the following sequence of hybrids:



► $\mathcal{L}_1 \equiv \mathcal{L}_{\text{hyb-1}}$; Without *accessing* the variable “bad”, the change can have no effect.

► $\mathcal{L}_{\text{hyb-1}} \approx \mathcal{L}_{\text{hyb-2}}$; By the bad-event lemma,

$$\left| \Pr [\mathcal{A} \diamond \mathcal{L}_{\text{hyb-1}} \Rightarrow 1] - \Pr [\mathcal{A} \diamond \mathcal{L}_{\text{hyb-2}} \Rightarrow 1] \right| \leq \Pr [\mathcal{A} \diamond \mathcal{L}_{\text{hyb-1}} \text{ sets bad} = 1] .$$

► $\mathcal{L}_{\text{hyb-2}} \equiv \mathcal{L}_2$; Regardless of input, the subroutine always returns false.

Hence

$$\mathcal{L}_1 \equiv \mathcal{L}_{\text{hyb-1}} \approx \mathcal{L}_{\text{hyb-2}} \equiv \mathcal{L}_2 \implies \mathcal{L}_1 \approx \mathcal{L}_2 .$$

3.4 Birthday Probabilities & Sampling With/out Replacement

Exercises

4.2. Which of the following are negligible functions in λ ? Justify your answers.

$$\frac{1}{2^{\lambda/2}}, \frac{1}{2^{\log(\lambda^2)}}, \frac{1}{\lambda^{\log(\lambda)}}, \frac{1}{\lambda^2}, \frac{1}{2^{(\log \lambda)^2}}, \frac{1}{(\log \lambda)^2}, \frac{1}{\lambda^{1/\lambda}}, \frac{1}{\sqrt{\lambda}}, \frac{1}{2^{\sqrt{\lambda}}}$$

Solution.

(1) $\frac{1}{2^{\lambda/2}}, \frac{1}{2^{\log(\lambda^2)}}, \frac{1}{\lambda^{\log(\lambda)}}, \frac{1}{\lambda^2}, \frac{1}{2^{(\log \lambda)^2}}, \frac{1}{(\log \lambda)^2}, \frac{1}{\sqrt{\lambda}}, \frac{1}{2^{\sqrt{\lambda}}}$ are negligible functions.

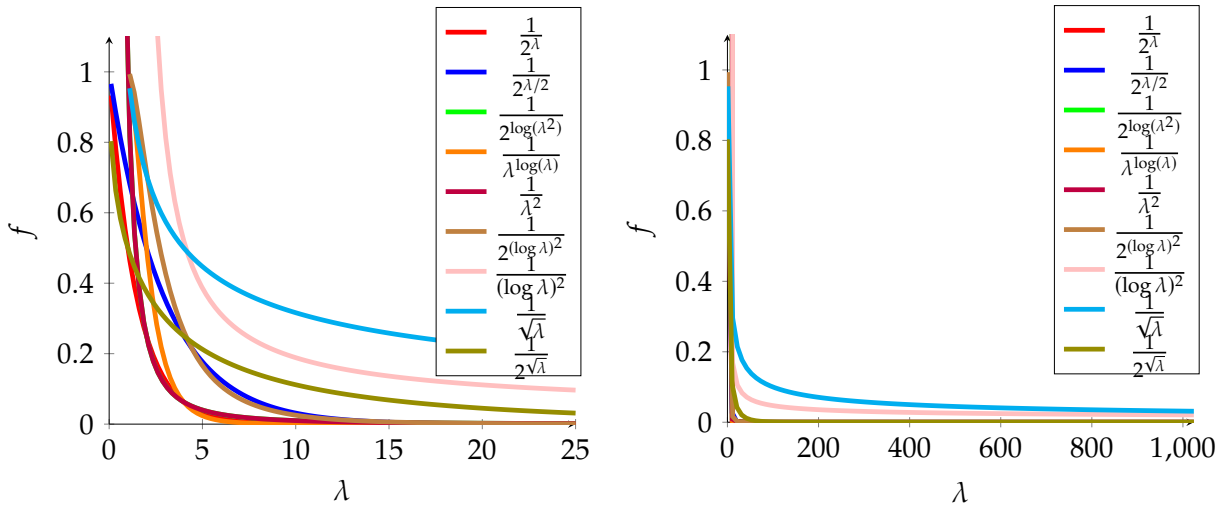


Figure 3.1: Negligible functions.

(2) $\frac{1}{\lambda^{1/\lambda}}$ is non-negligible functions.

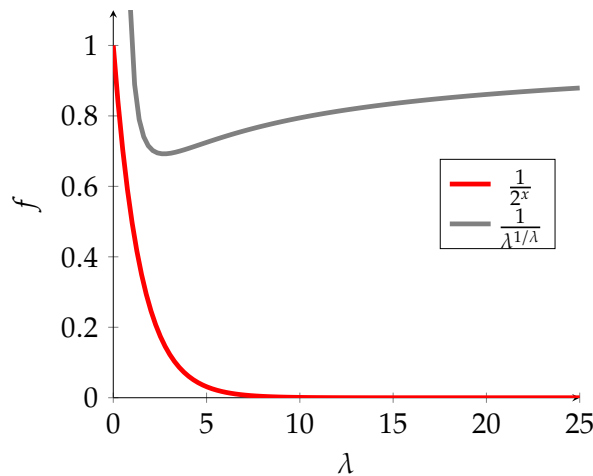


Figure 3.2: Non-negligible functions.

□

- 4.4. Show that when f is negligible, then for every polynomial p , the function $p(\lambda)f(\lambda)$ not only approaches 0, but it is also negligible itself.

Solution. We want to show that

$$p(\lambda)f(\lambda) \text{ is non-negligible} \implies f \text{ is non-negligible.}$$

Suppose that

$$\exists \text{ polynomial } q(\lambda) : \lim_{\lambda \rightarrow \infty} q(\lambda)p(\lambda)f(\lambda) = c \neq 0.$$

Then p is non-zero polynomial and f is non-zero function, and so

$$\lim_{\lambda \rightarrow \infty} q(\lambda) = \frac{c}{\lim_{\lambda \rightarrow \infty} p(\lambda)f(\lambda)} = \frac{c}{\text{constant}}.$$

Thus $\lim_{\lambda \rightarrow \infty} p(\lambda)f(\lambda)$ cannot be a zero. □

- 4.8. A deterministic program is one that uses no random choices. Suppose $\mathcal{L}_1$ and $\mathcal{L}_2$ are two deterministic libraries with a common interface. Show that either $\mathcal{L}_1 \equiv \mathcal{L}_2$, or else $\mathcal{L}_1$ & $\mathcal{L}_2$ can be distinguished with advantage 1.

Solution. Since both $\mathcal{L}_1$ and $\mathcal{L}_2$ are deterministic libraries, they will always produce the same output for the same input, i.e., either

$$\mathcal{L}_1(x) = \mathcal{L}_2(x) \quad \text{or} \quad \mathcal{L}_1(x) \neq \mathcal{L}_2(x)$$

for any input x .

(i) ($\mathcal{L}_1(x) = \mathcal{L}_2(x)$) Clearly,

$$(\forall \text{ input } x : \mathcal{L}_1(x) = \mathcal{L}_2(x)) \implies (\mathcal{L}_1 \equiv \mathcal{L}_2).$$

(ii) ($\mathcal{L}_1(x) \neq \mathcal{L}_2(x)$) Suppose that

$$\exists \text{ input } x : \mathcal{L}_1(x) \neq \mathcal{L}_2(x).$$

We construct an adversary $\mathcal{A}$ as follows:

- (a) $|\Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow 1] - \Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1]| = |1 - 0| = 1.$
- (b) $|\Pr[\mathcal{A} \diamond \mathcal{L}_1 \Rightarrow 1] - \Pr[\mathcal{A} \diamond \mathcal{L}_2 \Rightarrow 1]| = |0 - 1| = 1.$

□

4.12. Suppose you want to enforce password rules so that at least 2^{128} passwords satisfy the rules. How many characters long must the passwords be, in each of these cases?

- (a) Passwords consist of lowercase **a** through **z** only.
- (b) Passwords consist of lowercase and uppercase letters **a-z** and **A-Z**.
- (c) Passwords consist of lower/uppercase letters and digits **0-9**.
- (d) Passwords consist of lower/uppercase letters, digits, and any symbol characters that appear on a standard US keyboard (including the space character).

~	!	@	#	\$	%	^	&	*	(	)	_	+	Backspace
Tab	Q	W	E	R	T	Y	U	I	O	P	{	}	\
Caps Lock	A	S	D	F	G	H	J	K	L	:	"	'	Enter
Shift	Z	X	C	V	B	N	M	<	>	?	/	Shift	
Ctrl	Win	Alt								Alt	Win	Menu	Ctrl

Figure 3.3: Standard US Keyboard (<https://kbd-intl.narod.ru/english/layouts>)

Solution. We want to create a password system that allows for at least 2^{128} (16 bytes) different passwords.

- (a) We are only using lowercase letters **a-z**, which gives us **26** different possibilities for each character in the password. We need to solve the following equation for n (the length of the password):

$$26^n \geq 2^{128}.$$

Then

$$n \log(26) \geq 128 \log(2) \implies n \geq \frac{128 \cdot \log(2)}{\log(26)} \approx 27.2.$$

- (b)

$$52^n \geq 2^{128} \implies n \geq \frac{128 \cdot \log(2)}{\log(52)} \approx 22.4.$$

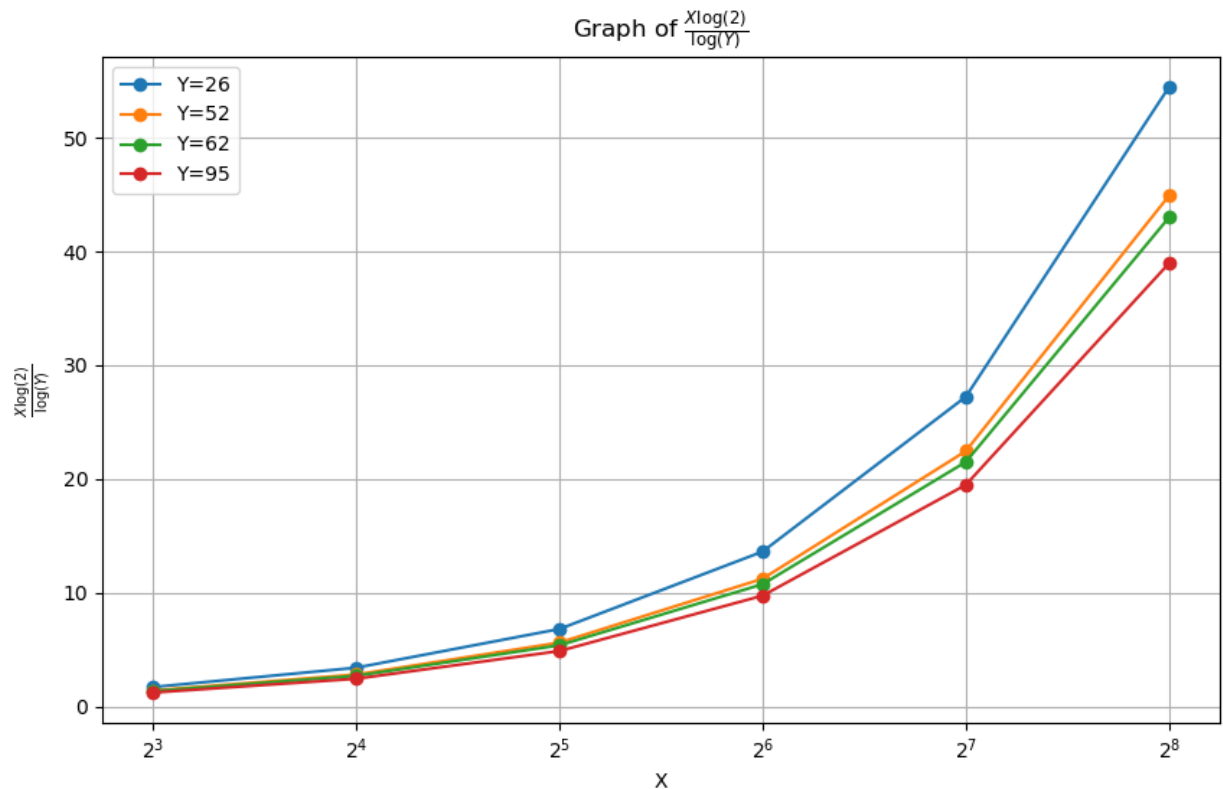
- (c)

$$62^n \geq 2^{128} \implies n \geq \frac{128 \cdot \log(2)}{\log(62)} \approx 21.5$$

- (d)

$$95^n \geq 2^{128} \implies n \geq \frac{128 \cdot \log(2)}{\log(95)} \approx 19.5$$

□



```

1 import matplotlib.pyplot as plt
2
3 # Given values of X and Y
4 X_values = [8, 16, 32, 64, 128, 256]
5 Y_values = [26, 52, 62, 95]
6
7 # Initialize a plot
8 plt.figure(figsize=(10,6))
9
10 # Loop through each Y value
11 for Y in Y_values:
12     # Calculate the expression for each X value
13     Z = [x * log(2) / log(Y) for x in X_values]
14
15     # Plot the result
16     plt.plot(X_values, Z, label='Y=' + str(Y), marker='o')
17
18 # Labeling the plot
19 plt.title(r'Graph of  $\frac{X \log(2)}{\log(Y)}$ ')
20 plt.xlabel('X')
21 plt.ylabel(r' $\frac{X \log(2)}{\log(Y)}$ ')
22 plt.xscale("log", base=2) # for logarithmic scale on x-axis
23 plt.legend()
24 plt.grid(True)
25 plt.show()

```

Chapter 4

Pseudo-random Generators (PRG)

4.1 Definition

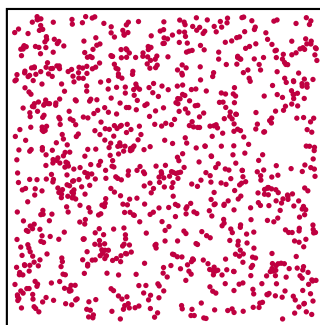
Pseudorandom Generator (PRG)

Definition 4.1. A deterministic function $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+l}$ with $l > 0$ is a **secure pseudorandom generator (PRG)** if $\mathcal{L}_{\text{PRG-real}}^G \approx \mathcal{L}_{\text{PRG-rand}}^G$, where:

$\mathcal{L}_{\text{PRG-real}}^G$	$\mathcal{L}_{\text{PRG-rand}}^G$
Query(): $s \leftarrow \{0, 1\}^\lambda$ return $G(s)$	Query(): $r \leftarrow \{0, 1\}^{\lambda+l}$ return r

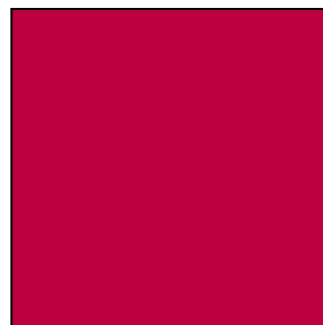
Remark 4.1. The value l is called the **stretch** of the PRG. The input s to the PRG is called a **seed**.

Remark 4.2. We illustrate the distributions, for a **length doubling** ($l = \lambda$) PRG (not drawn to scale):



$$\{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$$

Pseudorandom dist.



$$\{0, 1\}^{2\lambda}$$

Uniform dist.

Example 4.1 (Length-Doubling PRG). A straightforward approach for the PRG might be to duplicate its input string.

$$\boxed{\begin{array}{l} G(s): \\ \text{return } s \parallel s \end{array}}$$

For example, the following strings look likely they were sampled uniformly from $\{0, 1\}^8$:

11011101, 01110111, 01000100, ...

We can formalize this observation as an attack against the PRG-security of G :

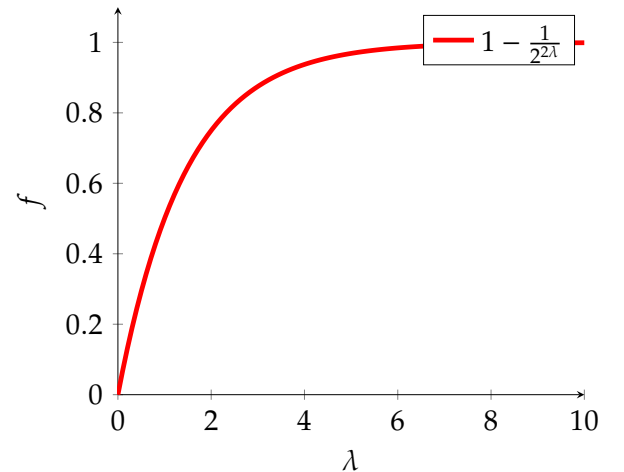
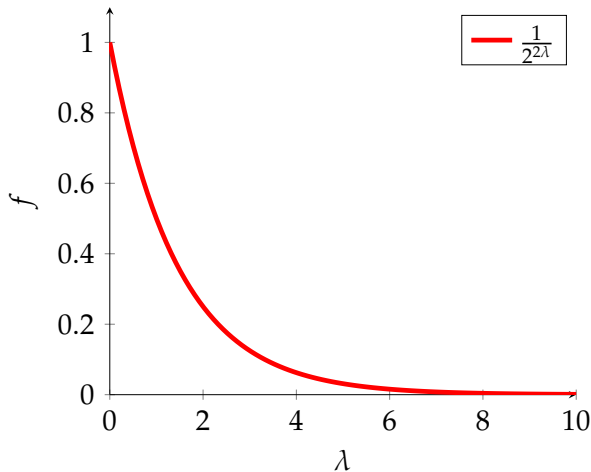
$$\boxed{\begin{array}{c} \mathcal{A} \\ x \parallel y := \text{Query}() \\ \text{return } x \stackrel{?}{=} y \end{array}}$$

Then Thus,

$$\begin{array}{c} \boxed{\begin{array}{c} \mathcal{A} \\ x \parallel y := \text{Query}() \\ \text{return } x \stackrel{?}{=} y \end{array}} \diamond \boxed{\begin{array}{c} \mathcal{L}_{\text{PRG-real}}^G \\ \text{Query}(): \\ s \leftarrow \{0, 1\}^\lambda \\ \text{return } G(s) \end{array}} \parallel \Pr[\mathcal{A} \diamond \mathcal{L}_{\text{PRG-real}}^G \Rightarrow 1] = 1. \\ \\ \boxed{\begin{array}{c} \mathcal{A} \\ x \parallel y := \text{Query}() \\ \text{return } x \stackrel{?}{=} y \end{array}} \diamond \boxed{\begin{array}{c} \mathcal{L}_{\text{PRG-rand}}^G \\ \text{Query}(): \\ r \leftarrow \{0, 1\}^{2\lambda} \\ \text{return } r \end{array}} \parallel \Pr[\mathcal{A} \diamond \mathcal{L}_{\text{PRG-rand}}^G \Rightarrow 1] = \frac{1}{2^{2\lambda}}. \end{array}$$

$$\text{Adv}_{\mathcal{A}} = \left| \Pr[\mathcal{A} \diamond \mathcal{L}_{\text{PRG-real}}^G \Rightarrow 1] - \Pr[\mathcal{A} \diamond \mathcal{L}_{\text{PRG-rand}}^G \Rightarrow 1] \right| = 1 - \frac{1}{2^{2\lambda}}$$

is non-negligible.



Comparison of Normalized Distributions:

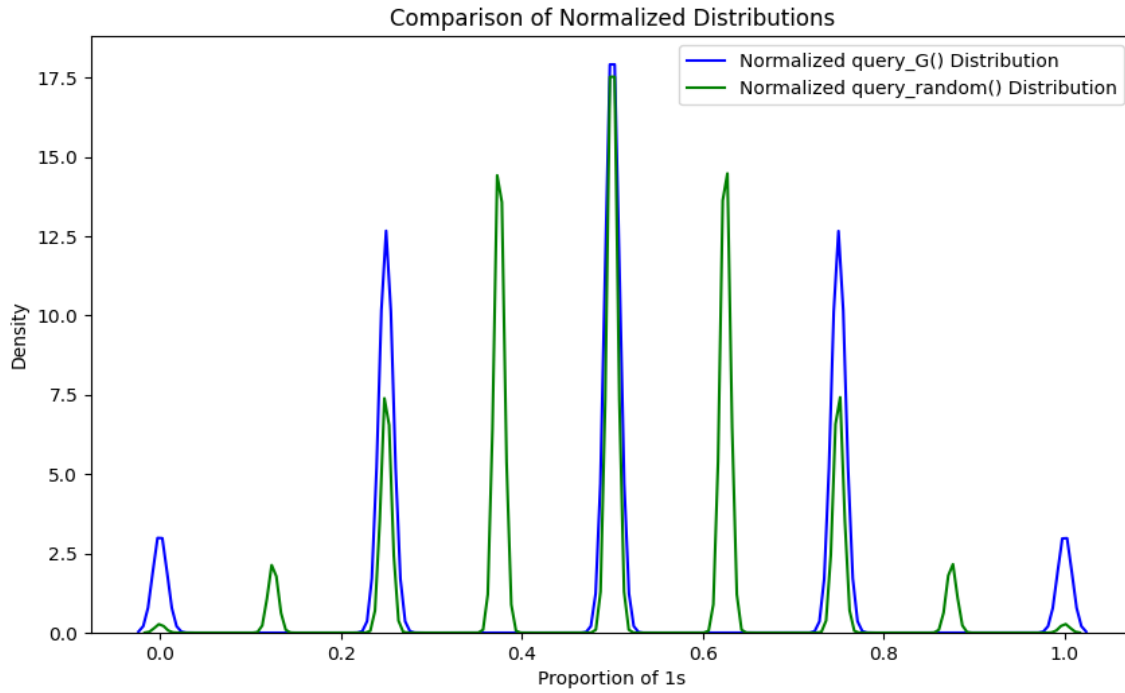


Figure 4.1: $\lambda = 4$, i.e., $\{0, 1\}^8$ with 1,000,000 experiments

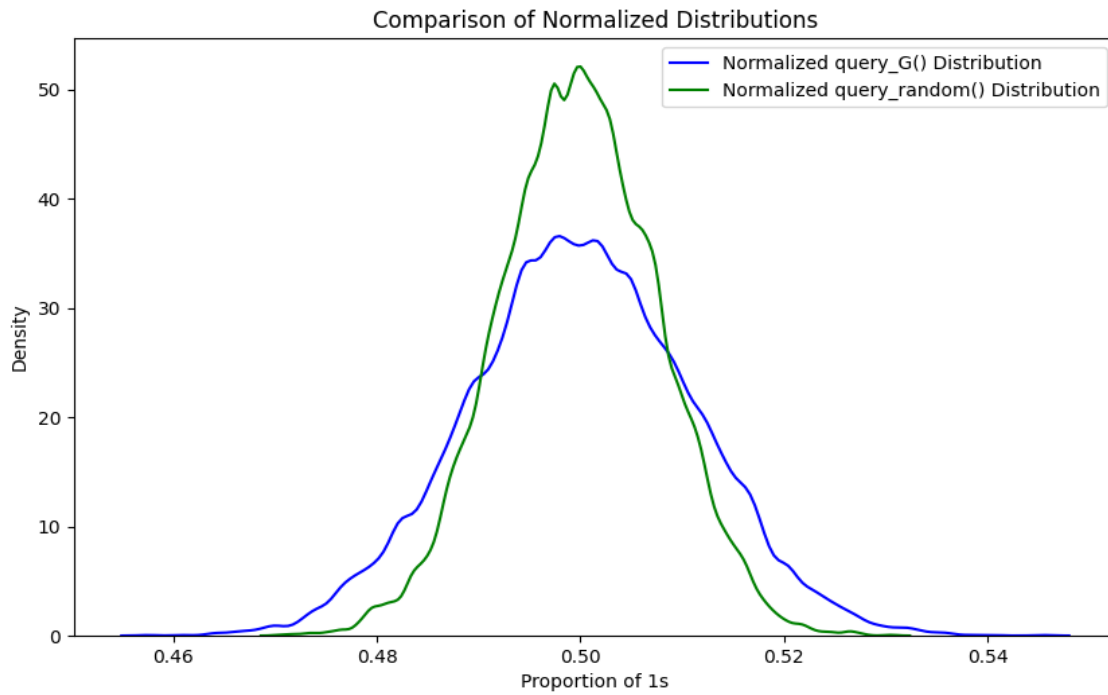


Figure 4.2: $\lambda = 64$, i.e., $\{0, 1\}^{128}$ with 100,000 experiments

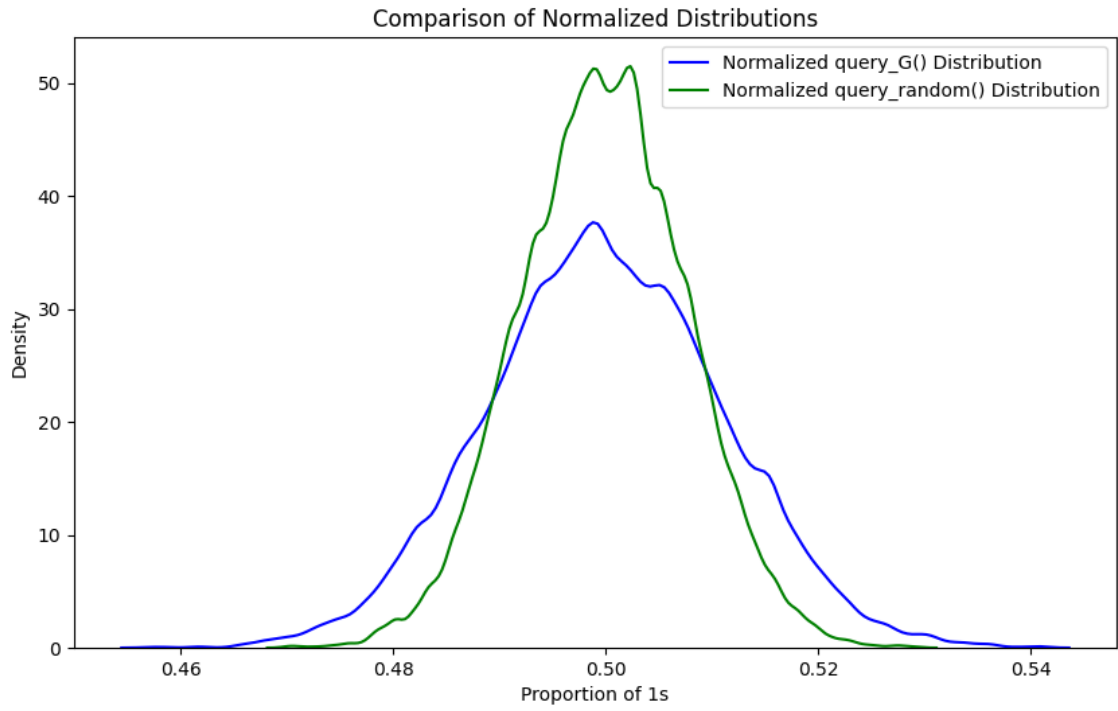


Figure 4.3: $\lambda = 256$, i.e., $\{0, 1\}^{512}$ with 100,000 experiments

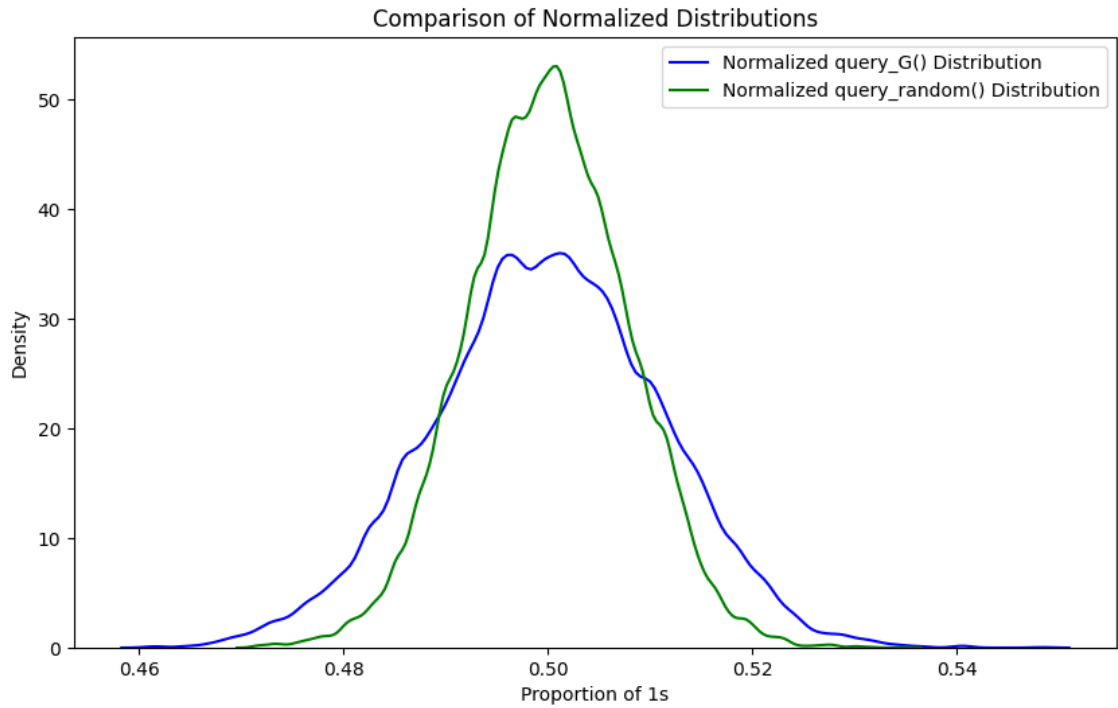


Figure 4.4: $\lambda = 1024$, i.e., $\{0, 1\}^{2048}$ with 100,000 experiments

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 def G(s):
6     """
7     This function takes a list of bits (s), and returns a new list where each 8-
8     bit unit is doubled.
9     """
10    return s * 2
11
12 def query_G():
13     lambda_length = 32
14     s = np.random.randint(0, 2, lambda_length).tolist()
15     return G(s)
16
17 def query_random():
18     lambda_length = 1024
19     l_length = 1024
20     # Generates a list of 0s and 1s
21     r = np.random.randint(0, 2, lambda_length + l_length).tolist()
22     return r
23
24 # Define the number of experiments to run
25 num_experiments = 100000
26
27 # Record the outputs
28 outputs_G = [query_G() for _ in range(num_experiments)]
29 outputs_random = [query_random() for _ in range(num_experiments)]
30
31 # Convert outputs to the sum of their elements to see the distribution of the
32 # number of 1s
33 sums_G = [sum(output) for output in outputs_G]
34 sums_random = [sum(output) for output in outputs_random]
35
36 # Normalizing the sums by the length of the binary string
37 norm_sums_G = [s / 2048 for s in sums_G]
38 norm_sums_random = [s / 2048 for s in sums_random]
39
40 # Generate a Kernel Density Estimate plot for each normalized distribution
41 plt.figure(figsize=(10, 6))
42
43 # Plot KDE for normalized sums_G
44 sns.kdeplot(norm_sums_G, bw_adjust=0.5, label='Normalized query_G() Distribution', color='blue')
45
46 # Plot KDE for normalized sums_random
47 sns.kdeplot(norm_sums_random, bw_adjust=0.5, label='Normalized query_random() Distribution', color='green')
48
49 # Add a legend and titles
50 plt.legend()
51 plt.title('Comparison of Normalized Distributions')
52 plt.xlabel('Proportion of 1s')
53 plt.ylabel('Density')
54
55 plt.show()

```

4.2 Shorter Keys in One-Time-Secret Encryption

One-time Pad (OTP)

Construction 4.1. The **one-time pad** are given below:

$\mathcal{K} = \{0, 1\}^\lambda$	<u>KeyGen :</u>	<u>Enc($k, m \in \{0, 1\}^\lambda$) :</u>	<u>Dec($k, c \in \{0, 1\}^\lambda$) :</u>
$\mathcal{M} = \{0, 1\}^\lambda$	$k \xleftarrow{\$} \mathcal{K}$	return $k \oplus m$	return $k \oplus c$
$\mathcal{C} = \{0, 1\}^\lambda$	return k		

Pseudo-OTP

Construction 4.2. Let $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{\lambda+l}$ be a PRG, and define the following:

$\mathcal{K} = \{0, 1\}^\lambda$	<u>KeyGen :</u>	<u>Enc(k, m) :</u>	<u>Dec(k, c) :</u>
$\mathcal{M} = \{0, 1\}^{\lambda+l}$	$k \xleftarrow{\$} \mathcal{K}$	return $G(k) \oplus m$	return $G(k) \oplus c$
$\mathcal{C} = \{0, 1\}^{\lambda+l}$	return k		

Computational One-Time Secrecy

Definition 4.2. An encryption scheme Σ has **(computational) one-time secrecy** if $\mathcal{L}_{\text{ots-1}}^\Sigma \approx \mathcal{L}_{\text{ots-2}}^\Sigma$. That is, if for all polynomial-time distinguishers $\mathcal{A}$, we have

$$\Pr \left[\mathcal{A} \diamond \mathcal{L}_{\text{ots-1}}^\Sigma \Rightarrow 1 \right] \approx \Pr \left[\mathcal{A} \diamond \mathcal{L}_{\text{ots-2}}^\Sigma \Rightarrow 1 \right].$$

Remark 4.3. Σ has **one-time secrecy** if

$\mathcal{L}_{\text{ots-1}}^\Sigma$		$\mathcal{L}_{\text{ots-2}}^\Sigma$
<u>Eve($m_1, m_2 \in \Sigma.\mathcal{M}$):</u> $k \leftarrow \Sigma.\text{KeyGen}$ $c \leftarrow \Sigma.\text{Enc}(k, m_1)$ return c	$\equiv$	<u>Eve($m_1, m_2 \in \Sigma.\mathcal{M}$):</u> $k \leftarrow \Sigma.\text{KeyGen}$ $c \leftarrow \Sigma.\text{Enc}(k, m_2)$ return c

Theorem 4.1. Let pOTP denote [Construction 4.2](#). If one constructs the pOTP utilizing a secure pseudorandom generator G , then pOTP has computational one-time secrecy.

Proof. We must show that

$$\mathcal{L}_{\text{ots-1}}^{\text{pOTP}} \approx \mathcal{L}_{\text{ots-2}}^{\text{pOTP}}.$$

We will show that a sequence of hybrid libraries satisfying the following:

$$\mathcal{L}_{\text{ots-1}}^{\text{pOTP}} \equiv \mathcal{L}_{\text{hyp-1}} \approx \mathcal{L}_{\text{hyp-2}} \equiv \mathcal{L}_{\text{hyp-3}} \equiv \mathcal{L}_{\text{hyp-4}} \equiv \mathcal{L}_{\text{hyp-5}} \approx \mathcal{L}_{\text{hyp-6}} \equiv \mathcal{L}_{\text{ots-2}}^{\text{pOTP}}.$$

$$\mathcal{L}_{\text{ots-1}}^{\text{pOTP}}: \quad \begin{array}{c} \mathcal{L}_{\text{ots-1}}^{\text{pOTP}} \\ \hline \text{Eve}(m_1, m_2 \in \{\mathbf{0}, \mathbf{1}\}^{\lambda+l}): \\ \hline k \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda} \\ c := G(k) \oplus m_1 \\ \text{return } c \end{array}$$

$\mathcal{L}_{\text{hyp-1}}:$

$$\begin{array}{c} \text{Eve}(m_1, m_2): \\ \hline z \leftarrow \text{Query}() \\ c := z \oplus m_1 \\ \text{return } c \end{array} \quad \diamond \quad \begin{array}{c} \mathcal{L}_{\text{PRG-real}}^G \\ \hline \text{Query}(): \\ \hline s \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda} \\ \text{return } G(s) \end{array}$$

$\mathcal{L}_{\text{hyp-2}}:$

$$\begin{array}{c} \text{Eve}(m_1, m_2): \\ \hline z \leftarrow \text{Query}() \\ c := z \oplus m_1 \\ \text{return } c \end{array} \quad \diamond \quad \begin{array}{c} \mathcal{L}_{\text{PRG-rand}}^G \\ \hline \text{Query}(): \\ \hline r \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda+l} \\ \text{return } r \end{array}$$

$$\mathcal{L}_{\text{hyp-3}}: \quad \begin{array}{c} \mathcal{L}_{\text{ots-1}}^{\text{OTP}} \\ \hline \text{Eve}(m_1, m_2): \\ \hline z \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda+l} \\ c := z \oplus m_1 \\ \text{return } c \end{array}$$

$$\mathcal{L}_{\text{ots-2}}^{\text{pOTP}}: \quad \begin{array}{c} \mathcal{L}_{\text{ots-2}}^{\text{pOTP}} \\ \hline \text{Eve}(m_1, m_2 \in \{\mathbf{0}, \mathbf{1}\}^{\lambda+l}): \\ \hline k \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda} \\ c := G(k) \oplus m_2 \\ \text{return } c \end{array}$$

$\mathcal{L}_{\text{hyp-6}}:$

$$\begin{array}{c} \text{Eve}(m_1, m_2): \\ \hline z \leftarrow \text{Query}() \\ c := z \oplus m_2 \\ \text{return } c \end{array} \quad \diamond \quad \begin{array}{c} \mathcal{L}_{\text{PRG-real}}^G \\ \hline \text{Query}(): \\ \hline s \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda} \\ \text{return } G(s) \end{array}$$

$\mathcal{L}_{\text{hyp-5}}:$

$$\begin{array}{c} \text{Eve}(m_1, m_2): \\ \hline z \leftarrow \text{Query}() \\ c := z \oplus m_2 \\ \text{return } c \end{array} \quad \diamond \quad \begin{array}{c} \mathcal{L}_{\text{PRG-rand}}^G \\ \hline \text{Query}(): \\ \hline r \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda+l} \\ \text{return } r \end{array}$$

$$\mathcal{L}_{\text{hyp-4}}: \quad \begin{array}{c} \mathcal{L}_{\text{ots-2}}^{\text{OTP}} \\ \hline \text{Eve}(m_1, m_2): \\ \hline z \leftarrow \{\mathbf{0}, \mathbf{1}\}^{\lambda+l} \\ c := G(k) \oplus m_2 \\ \text{return } c \end{array}$$

□

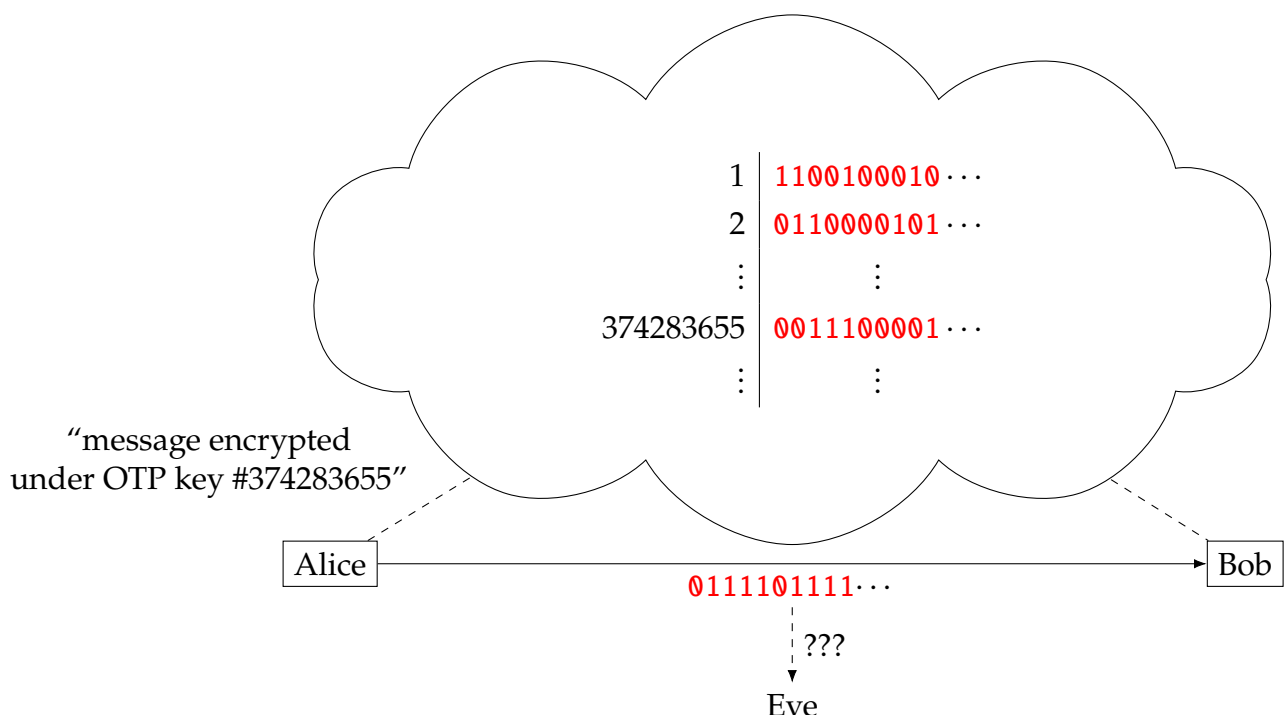
Chapter 5

Pseudo-Random Functions & Block Ciphers

Alice then informs Bob that chunk n has been used, without revealing the actual bits of r_n . Bob, having access to the shared randomness $\mathcal{R}$, can decrypt the message by computing:

$$m = c \oplus r_n$$

Since the eavesdropper does not possess $\mathcal{R}$, and given that each r_n is used only once, the encrypted message c reveals no information about the message m as long as the XOR operation with r_n is uniformly distributed. Thus, the encryption scheme is information-theoretically secure.

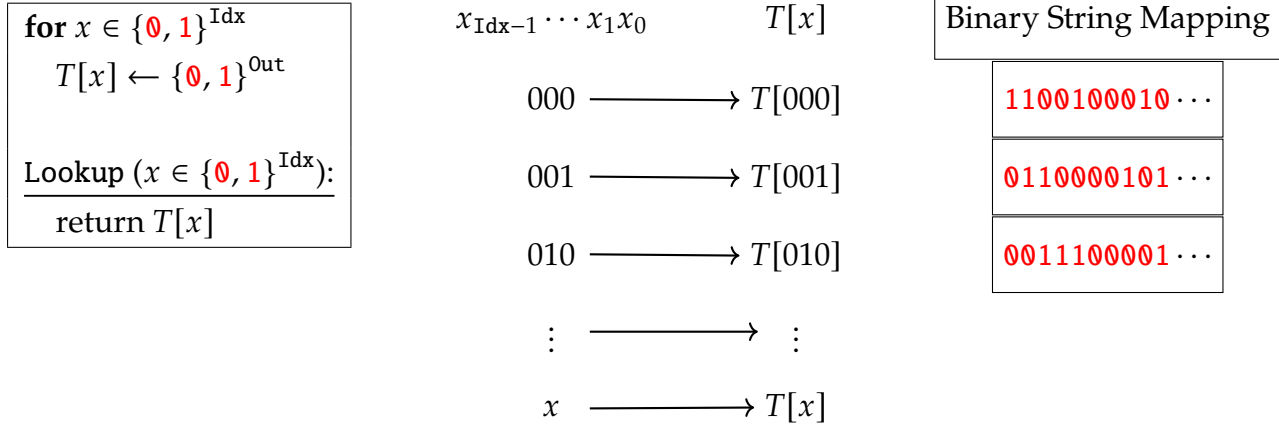


While the notion of infinite shared randomness is impractical, it can be approximated by an exponential amount of shared resources. Consider a table $\mathcal{T}$ shared between Alice and Bob, containing 2^λ unique one-time pad keys, sufficient for an extensive number of message encryptions.

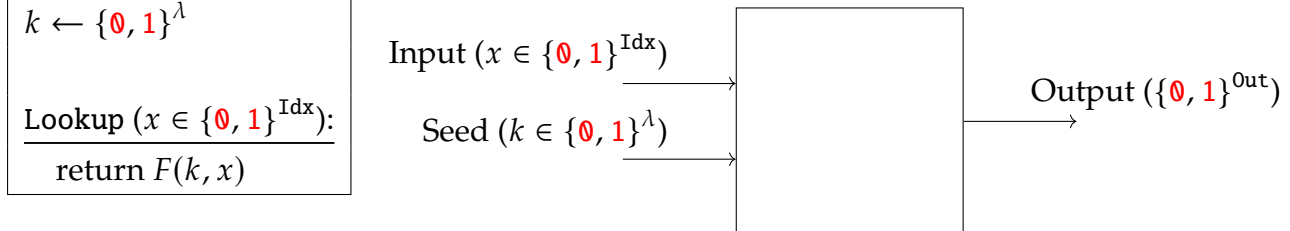
5.1 Definition

The goal of a pseudorandom function is to “look like” a uniformly chosen array / lookup table.

Array T



Pseudorandom Function F



A Pseudo-Random Function (PRF) is a fundamental concept in cryptography, typically defined in the context of a family of functions. Let’s denote a PRF family by F , where each function f_k in F is indexed by a key k from a key space K . The function f_k maps inputs from an input space X to outputs in an output space Y . Mathematically, for a key k , the PRF is defined as:

$$f_k : X \rightarrow Y$$

Given k , for any input $x \in X$, $f_k(x)$ is easy to compute. However, without knowledge of k , the function’s output is indistinguishable from a truly random function from an adversary’s point of view, given polynomially bounded computational resources.

A truly random function (RF), on the other hand, is a function where every possible input $x \in X$ is mapped to an output $y \in Y$ completely at random, without any deterministic process. Formally, an RF is defined as a function $h : X \rightarrow Y$ where each $h(x)$ is chosen uniformly at random from Y .

The key distinction between a PRF and an RF is that a PRF’s output is reproducible given the same key and input, whereas an RF provides no such guarantee—the output for the same input can vary with each function invocation.

PRF Security

Definition 5.1. A deterministic function

$$F : \{0, 1\}^\lambda \times \{0, 1\}^{\text{Idx}} \rightarrow \{0, 1\}^{\text{Out}}$$

is a **secure pseudo-random function (PRF)** if $\mathcal{L}_{\text{PRF-real}}^F \approx \mathcal{L}_{\text{PRF-rand}}^F$, where

$\mathcal{L}_{\text{PRF-real}}^F$
$k \leftarrow \{0, 1\}^\lambda$
<u>Lookup ($x \in \{0, 1\}^{\text{Idx}}$):</u>
return $F(k, x)$

$\mathcal{L}_{\text{PRF-rand}}^F$
$T := \{\}$
<u>Lookup ($x \in \{0, 1\}^{\text{Idx}}$):</u>
if $T[x]$ undefined :
$T[x] \leftarrow \{0, 1\}^{\text{Out}}$
return $T[x]$

Example 5.1 (How NOT to Build a PRF). Suppose we have a length-doubling PRG

$$G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2\lambda}$$

and try to use it to construct a PRF F as follows:

$F(k, x):$
$\text{return } G(k) \oplus x$

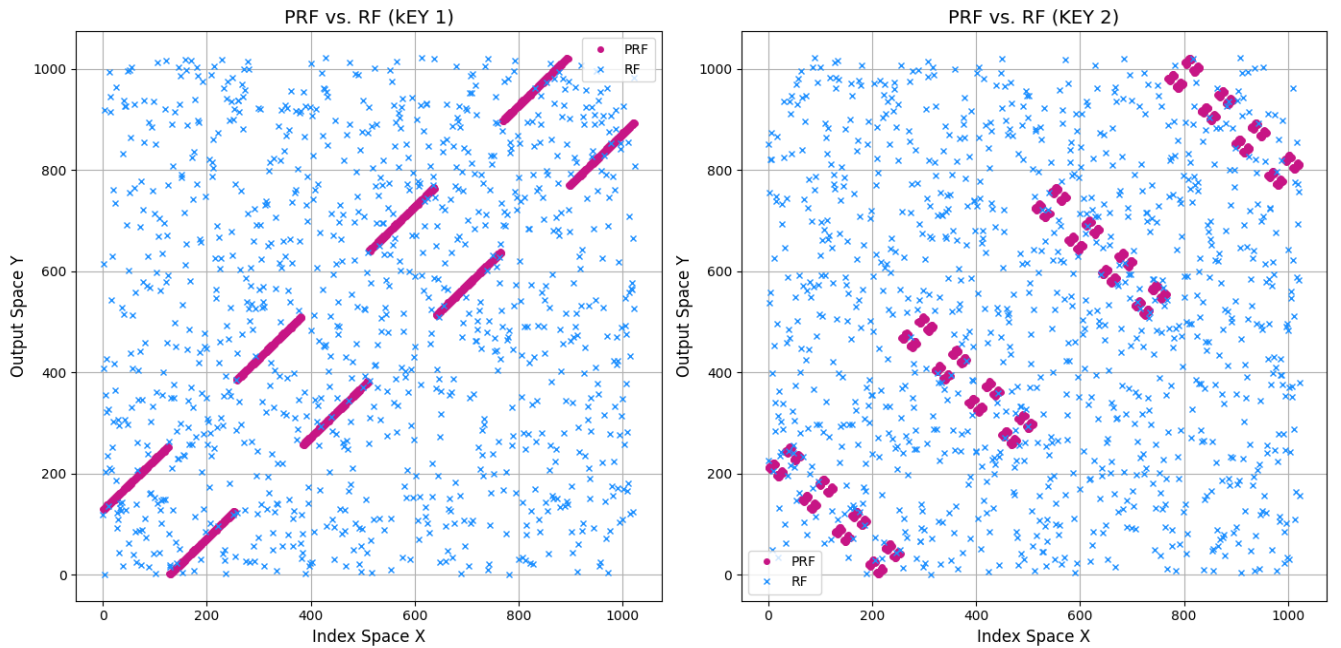


Figure 5.1: PRF vs RF

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Let's visualize the concept of a Pseudo-Random Function (PRF) using a simple
   example.
5
6 # Generate a random key
7 key1 = np.random.randint(0, 2**10, size=1)
8 key2 = np.random.randint(0, 2**10, size=1)
9
10 def prf(x, k):
11     # XOR the input with the key
12     x_k = np.bitwise_xor(x, k)
13     return x_k
14
15 # Define a truly random function (RF)
16 def rf(x, y_space):
17     # Choose a random output from the output space for each input
18     return np.random.choice(y_space)
19
20 # Define new input and output spaces of different sizes
21 input_space = np.arange(2**10)
22 output_space = np.arange(2**10)
23
24 # Compute the PRF and RF outputs for the smaller input/output space
25 prf_output1 = np.array([prf(x, key1) for x in input_space1])
26 rf_output1 = np.array([rf(x, output_space) for x in input_space])
27 prf_output2 = np.array([prf(x, key2) for x in input_space1])
28 rf_output2 = np.array([rf(x, output_space) for x in input_space])
29
30 # Create a figure with specified figure size
31 plt.figure(figsize=(14, 7))
32 # Subplot 1 for PRF vs RF comparison in the first scenario
33 plt.subplot(1, 2, 1)
34 plt.plot(input_space1, prf_output1, 'o', color='mediumvioletred', label='PRF',
   markersize=4)
35 plt.plot(input_space1, rf_output1, 'x', color='dodgerblue', label='RF',
   markersize=4)
36 plt.title('PRF vs. RF (KEY 1)', fontsize=14)
37 plt.xlabel('Index Space X', fontsize=12)
38 plt.ylabel('Output Space Y', fontsize=12)
39 plt.legend()
40 plt.grid(True)
41 # Subplot 2 for PRF vs RF comparison in the second scenario
42 plt.subplot(1, 2, 2)
43 plt.plot(input_space1, prf_output2, 'o', color='mediumvioletred', label='PRF',
   markersize=4)
44 plt.plot(input_space1, rf_output2, 'x', color='dodgerblue', label='RF',
   markersize=4)
45 plt.title('PRF vs. RF (KEY 2)', fontsize=14)
46 plt.xlabel('Index Space X', fontsize=12)
47 plt.ylabel('Output Space Y', fontsize=12)
48 plt.legend()
49 plt.grid(True)
50 # Adjust layout to prevent overlap and show the plot
51 plt.tight_layout()
52 plt.show()

```


Bibliography

- [1] M. Rosulek, *The Joy of Cryptography*, [Online]. Available: <https://joyofcryptography.com>
- [2] N. P. Smart, *Cryptography Made Simple*. 1st ed. Springer International Publishing, 2016.
- [3] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*. 2nd ed. Chapman and Hall/CRC, 2014.